KCPC Week 4 Reading
Introduction to Graphs: Traversals, Structure, and Modelling

Follow this location-stamped reading guide alongside the Workshop 4 slides and problem sheet. Each entry attributes the reference, specifies the exact chapters, sections, and page numbers covering the workshop's theory, and connects each reading directly to the corresponding problems. For primary historical and research literature, see *Week 4 Papers*.

I. Core Competitive Programming Textbooks

Competitive Programmer's Handbook (Antti Laaksonen)

Draft of July 3, 2018 (freely available online). https://cses.fi/book/book.pdf

- **Chapter 11: Basics of Graphs (§§11.1–11.4, pp. 109–116):** Introduces graph terminology (paths, cycles, connectivity, connected components, trees, directed vs. undirected graphs, vertex degrees, and bipartite colourings) and compares adjacency list ($O(V + E)$), adjacency matrix ($O(V^2)$), and edge list representations. Read this alongside Slide Sections 1–2.
- **Chapter 12: Graph Traversal (§§12.1–12.2, pp. 117–124):** Covers Depth-First Search (DFS), Breadth-First Search (BFS), shortest unweighted path computation, connected component enumeration, undirected cycle detection via edge/degree analysis, and bipartiteness testing (2-colouring). Directly supports Warm-Up Problems 1.1–1.4 (*Building Roads*, *Message Route*, *Round Trip*, *Building Teams*) and Core Problem 2.2 (*Cyclic Components*).
- **Chapter 14: Tree Algorithms (§§14.1–14.2, pp. 133–138):** Develops tree traversal properties and presents Algorithm 2 (§14.2, pp. 136–137): computing the diameter of a tree using two BFS/DFS traversals by finding the farthest node from an arbitrary root. Directly supports Problem 2.5 (*Tree Diameter*).
- **Chapter 16: Directed Graphs (§§16.1–16.3, pp. 151–158):** Covers topological sorting on Directed Acyclic Graphs (DAGs), dynamic programming on DAGs (counting the number of paths and finding longest/shortest paths), and successor/functional graphs in which every vertex has out-degree 1. Directly supports Problems 2.1, 2.3, 2.4, and 3.2 (*Badge*, *DAG Number of Routes*, *Longest Path*, and *Game on Axis*).

Competitive Programming 4, Book 1 (Steven Halim, Felix Halim, Suhendry Effendy)

Lulu Press / NUS, 2020. https://cpbook.net/

- **Chapter 4: Graph (§§4.1–4.2, pp. 169–192):** Provides concrete C++ and Python implementations of graph data structures, DFS, BFS, connected components, flood fill, topological sort (both DFS post-order and Kahn's algorithm), bipartite graph check, and cycle detection on directed and undirected graphs.
- **Chapter 4, §4.6: Special Graphs (pp. 222–231):** Explores how general hard problems become solvable in $O(V+E)$ linear time when restricted to trees, DAGs (counting paths, longest paths), or bipartite graphs, plus graph modelling techniques such as state-space expansion (Problem 3.1, *Rudolf and Subway*).

II. Formal Algorithmic Proofs and Invariants

Introduction to Algorithms, 3rd Edition (Cormen, Leiserson, Rivest, Stein)

MIT Press, 2009.

- **Chapter 22, §22.1: Representations of Graphs (pp. 589–593):** Formal analysis of adjacency lists vs. adjacency matrices, memory trade-offs, and operation complexities.
- **Chapter 22, §22.2: Breadth-First Search (pp. 594–602):** Rigorous proof of the FIFO queue distance invariant (Lemma 22.1–22.3 and Theorem 22.5), showing that BFS discovers every reachable vertex v at its exact shortest-path distance $\delta(s, v)$ and that the predecessor pointers $\pi[v]$ form a breadth-first shortest-path tree (Problem 1.2).
- **Chapter 22, §22.3: Depth-First Search (pp. 603–611):** Establishes discovery and finishing timestamps $(d[u], f[u])$, the Parenthesis Theorem (Theorem 22.7), the White-Path Theorem (Theorem 22.9), and edge classification (tree, back, forward, and cross edges), including Theorem 22.10 proving that every edge in an undirected DFS is either a tree edge or a back edge (Problem 1.3).
- **Chapter 22, §22.4: Topological Sort (pp. 612–615):** Proves Lemma 22.11 (a directed graph is acyclic iff DFS yields no back edges) and Theorem 22.12 (ordering vertices by decreasing finishing times produces a valid topological sort).

III. Online Algorithmic Encyclopaedias and Interactive Guides

CP-Algorithms (e-maxx.ru English Translation)

- **Breadth-First Search:** https://cp-algorithms.com/graph/breadth-first-search.html
  Queue algorithm, shortest unweighted path reconstruction, and connected components (Problems 1.1, 1.2, 3.1).
- **Depth-First Search:** https://cp-algorithms.com/graph/depth-first-search.html
  Entry/exit timers, edge classification, and cycle reconstruction (Problems 1.3, 2.1, 3.2; see also `/graph/finding-cycle.html`).
- **Bipartite Check:** https://cp-algorithms.com/graph/bipartite-check.html
  2-colouring theorem and BFS/DFS implementation (Problem 1.4).
- **Topological Sorting:** https://cp-algorithms.com/graph/topological-sort.html
  DAG ordering and dynamic programming on DAGs (Problems 2.3, 2.4).

USACO Guide (Silver & Gold Modules)

- **Graph Traversal (Silver):** https://usaco.guide/silver/graph-traversal
  Interactive tutorials and walkthroughs for connected components, *Building Roads*, and *Building Teams*.
- **BFS & Topological Sort (Gold):** https://usaco.guide/gold/bfs and https://usaco.guide/gold/toposort
  Shortest-path parent tracking, DAG dynamic programming (*Game Routes*, *Longest Path*), and auxiliary graph modelling.

IV. Suggested Study Order

Read Laaksonen §§11.1–11.4 and §§12.1–12.2 (or CP-Algorithms on BFS/DFS/Bipartite) and complete the Warm-Up Exercises (Problems 1.1–1.4). Next, read Laaksonen §§14.1–14.2 and §§16.1–16.3 alongside CLRS §§22.3–22.4 before tackling the Core Practice Set (Problems 2.1–2.5). Finally, use Halim §4.6 on auxiliary state-space graphs and functional graph structure to solve the Extended Homework (Problems 3.1–3.2).